Project work part 2

- Author: Mehdi Jamali

Links

- GitHub: Click here
- Streamlit: Click here
- Video: Click here

Declaration of AI usage

The following AI tools were used during the development of this project:

- **GitHub Copilot:**
  - Used for code completion, suggestions, and feedback during development.
- **Gemini:**
  - Used for coding assistance and for finding relevant functions and methods.
- **NotebookLM (Gemini):**
  - Used as a research assistant to answer questions and explain topics related to the project.

The suggestions were reviewed, adapted, and tested before they were used in the project.

The AI tools were used within the guidelines of "K3 - Full use of AI" at NMBU.

Work Log:

I started with branching the repo on github and made a branch called ProjectPart2, then i opened the branch on github desktop and to see if it was right in VsCode i used the terminal command `git branch --show-current` but gir was'nt available in the terminal so i used `Get-content .git\HEAD` which showed `ref: refs/heads/ProjectPart2` to confrim i was on the right branch.

I updated my code to comply with the suggestions from Abdirahman from the 1st assignment. An important problem occurred when the application could no longer find `reservoirs.csv` after the folder structure was changed. Instead of continuing to depend on the CSV file, I created a `nve_data.py` module that retrieves reservoir observations directly from the public NVE Magasinstatistikk API. The API connection was tested successfully with HTTP status 200, and it returned 14,922 observations. The returned data included the expected area types `EL`, `NO`, and `VASS`.

I also added local Streamlit secrets support. Secret values are stored in `.streamlit/secrets.toml`, while the file is excluded through `.gitignore`. This prevents API tokens from being committed to GitHub. The NVE API does not require a secret. The ENTSO-E security token will be read through Streamlit secrets when the ENTSO-E integration is implemented.

During the project, I used AI assistance mainly for solving larger technical problems and understanding unfamiliar tools. One major problem was the change in the folder structure, which caused the Streamlit pages and imports to stop working. AI helped me trace the file paths, update the navigation, and organise the Part 1 and Part 2 pages correctly. Another major problem was building the data pipeline with Cassandra, Spark, and MongoDB. I had limited previous experience with these technologies, and several errors occurred, including an invalid ENTSO-E token, incorrect MongoDB connection details, missing Java configuration for Spark, and Windows Hadoop warnings. AI helped me understand the error messages and suggested steps for testing each part separately. I then implemented and tested the changes myself.

Finally, I structured the Streamlit page with columns, radio buttons, month pills, charts, and an expander explaining the data source. I tested the page locally and verified that the data could be loaded from MongoDB, that the connection and year selections worked, and that the pie chart and line plot displayed the processed electricity-flow data correctly.

Part 2 data pipeline

The Part 2 project combines API connections, databases, data processing and Streamlit. The data flow is ENTSO-E API to Cassandra to Spark to MongoDB to Streamlit. The following sections document the processed data and recreate the required plots.

Library imports

We start by importing the libraries needed for reading the processed data and creating the plots.

In [1]:

```python
# Import Path so the notebook can find the processed data file.
from pathlib import Path

# Import Pandas for reading and inspecting the processed data.
import pandas as pd

# Import Matplotlib so the figures are embedded as visible PNG output in exports.
import matplotlib.pyplot as plt
```

Read the processed electricity data

The raw electricity observations were stored in Cassandra and processed with Spark. Spark created a smaller JSON file with monthly totals, which is loaded here for the notebook visualisations.

In [2]:

```python
# Use the path from the project root when the notebook is opened there.
relative_data_path = Path("data") / "processed" / "electricity_flows" / "monthly_flows.json"

# Use the parent folder when the notebook is opened from Project Notebooks.
processed_path = relative_data_path if relative_data_path.exists() else Path("..") / relative

# Read the monthly Spark result into a Pandas dataframe.
monthly_flows = pd.read_json(processed_path)

# Check the size and contents of the processed dataset.
print(f"Processed monthly documents: {len(monthly_flows)}")
print("Connections:", sorted(monthly_flows["connection"].unique()))
print("Years:", sorted(monthly_flows["year"].unique()))
monthly_flows.head()
```

```
Processed monthly documents: 60
Connections: ['DK1-NO2', 'SE2-NO3', 'SE3-NO4']
Years: [np.int64(2024), np.int64(2025), np.int64(2026)]
```

Out[2]:

	connection	year	month	direction	total_energy_mwh
0	DK1-NO2	2025	1	Into Norway	762042.750
1	DK1-NO2	2025	1	Out of Norway	91769.725
2	SE2-NO3	2024	10	Into Norway	5472.210
3	SE2-NO3	2024	10	Out of Norway	136564.910
4	SE2-NO3	2024	11	Into Norway	70336.190

Annual import and export pie chart

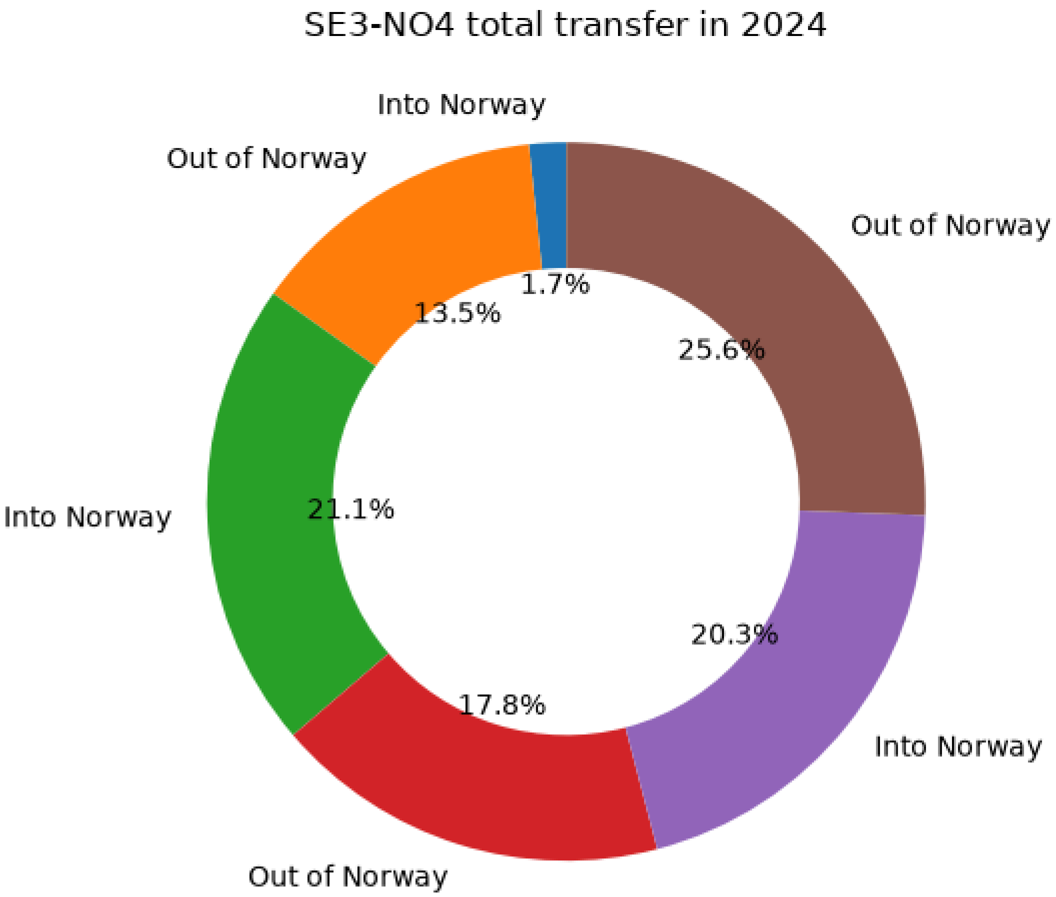
The assignment requires a pie chart for one year and one chosen connection. The two slices represent electricity transferred into Norway and electricity transferred out of Norway.

In [3]:

```
# Select the main connection and the first available year for the pie chart.
chosen_connection = "SE3-NO4"
chosen_year = int(monthly_flows[monthly_flows["connection"] == chosen_connection]["year"].min

# Keep only the selected connection and year.
chosen_data = monthly_flows[
    (monthly_flows["connection"] == chosen_connection)
    & (monthly_flows["year"] == chosen_year)
]

# Create a pie chart comparing the two transfer directions.
fig, ax = plt.subplots(figsize=(7, 5))
ax.pie(
    chosen_data["total_energy_mwh"],
    labels=chosen_data["direction"],
    autopct="%.1f%%",
    startangle=90,
    wedgeprops={"width": 0.35},
)
ax.set_title(f"{chosen_connection} total transfer in {chosen_year}")
plt.tight_layout()
plt.show()
```



Monthly transfer line plot

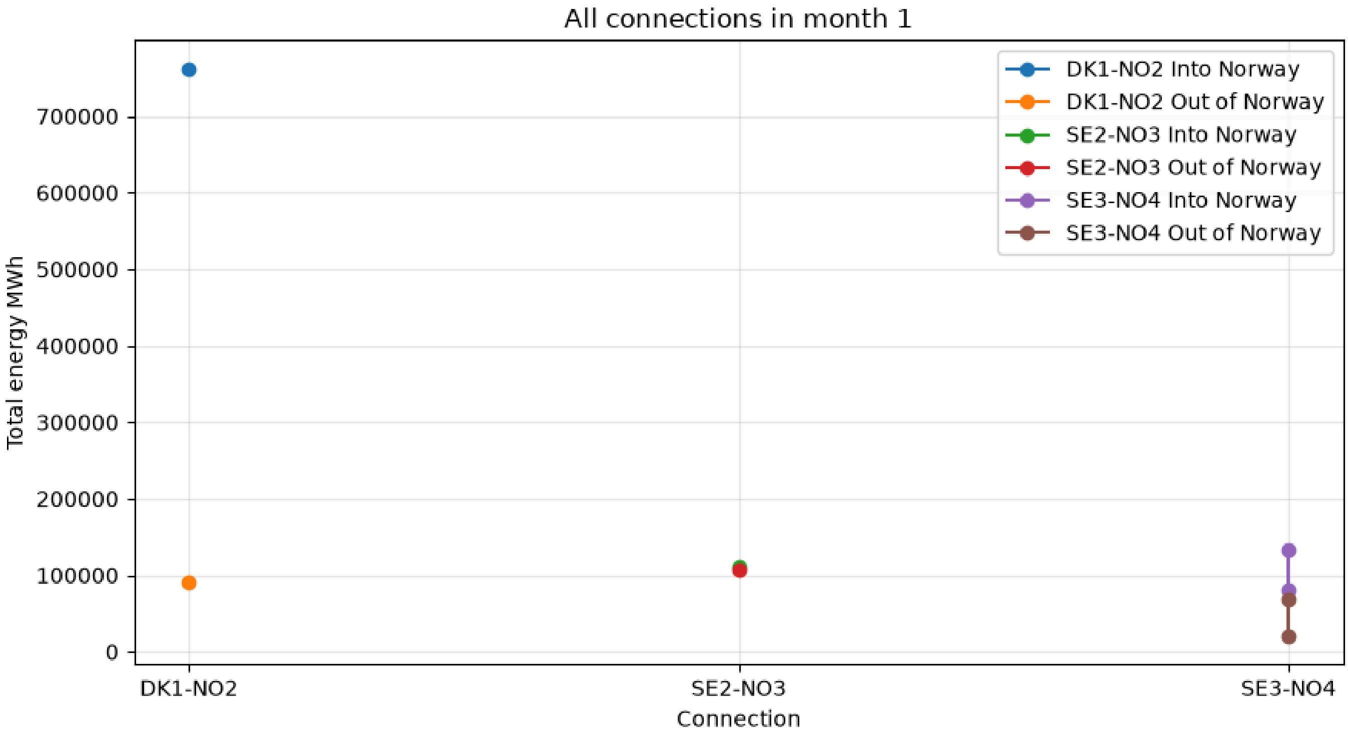
The assignment also requires a line plot for the first month of a year. The plot below compares all verified connections and separates the two transfer directions.

In [4]:

```python
# Select the first month available in the processed dataset.
first_month_number = int(monthly_flows["month"].min())
first_month = monthly_flows[monthly_flows["month"] == first_month_number].copy()

# Create a separate series for each connection and transfer direction.
first_month["series"] = first_month["connection"] + " " + first_month["direction"]

# Create a line plot for the selected month.
fig, ax = plt.subplots(figsize=(9, 5))
for series_name, series_data in first_month.groupby("series"):
    ax.plot(
        series_data["connection"],
        series_data["total_energy_mwh"],
        marker="o",
        label=series_name,
    )
ax.set_title(f"All connections in month {first_month_number}")
ax.set_xlabel("Connection")
ax.set_ylabel("Total energy MWh")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Result and limitation

The processed data is also used by the MongoDB page in the Streamlit application. SE3 NO4 is the main connection and contains the complete two year dataset. SE2 NO3 and DK1 NO2 are included as verified comparison connections. Loading a complete two year dataset for every connection took too long because the ENTSO E API had to be requested repeatedly for several periods and directions.